

《面向对象程序设计》之——

类与对象及其应用

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

讲授内容

- 抽象数据类型的概念
- 类的特点和作用
- 类的定义
- 类的访问控制
- 类的数据成员和成员函数
- 类的静态成员
- 类的对象
- 类的构造函数和析构函数
- 类的复合
- this指针
- const对象/成员函数
- 友元函数
- 友元类

什么是抽象数据类型？

- 对问题空间的实体进行抽象
 - 包含哪些属性（数据）和功能（函数）
 - 抽象的结果——对数据和函数的封装
- 类(class)——是抽象数据类型，实现了对数据和函数的封装（必须通过接口函数才能操作数据）

例子1: 用结构实现栈

```
#include <iostream.h>
struct Stack                //结构定义
{int *data;                //栈数据存储
  int memNum;              //栈元素个数
  int size;                //栈大小
};
int initStack(Stack &s,int size);
void delStack(Stack &s);
int popStack(Stack &s, int &num);
int pushStack(Stack &s, int mem);
```

//初始化栈

```
int initStack(Stack &s, int size)
{
    s.data = new int[size];
    if (s.data == NULL)
        return 0;
    s.memNum = 0;
    s.size = size;
    return 1;
}
```

//释放栈

```
void delStack(Stack &s)
{
    delete [] s.data;
}
```

//弹栈，无数据时返回0，否则返回1

```
int popStack(Stack &s, int &num)
{if (s.memNum == 0)
    return 0;
    num = s.data[--s.memNum] ;
    return 1;
}
```

//压栈，成功则返回1，否则返回0

```
int pushStack(Stack &s, int mem)
{if (s.memNum == s.size)
    return 0;
    s.data[s.memNum++] = mem;
    return 1;
}
```

```
main()
{int i, num;
  Stack newStack;
  initStack(newStack, 10);
  cout<<"Push integers to stack :\"
    <<endl;
  for (i=0;i<10;i++)
  { cout<<i<<" ";
    pushStack(newStack, i);
  }
  cout<<endl;
  cout<<"From function popStack :\"<<endl
  for (i=0; i<10; i++)
    if (popStack(newStack, num))
      cout<<num<<" ";
  cout<<endl<<endl;
```

```
for (i=10;i<20;i++)
    newStack.data[newStack.memNum++] = i;
cout<<"Reading from struct newStack : "
    <<endl;
for (i=0;i<10;i++)
    cout<<newStack.data[i]<<" ";
cout<<endl;
for (i=10;i<20;i++)
    cout<<newStack.data[i]<<" ";
cout<<endl;
delStack(newStack);
return 0;
}
```


程序运行结果:

Push integers to stack :

0 1 2 3 4 5 6 7 8 9

From function popStack :

9 8 7 6 5 4 3 2 1 0

Reading from struct newStack :

10 11 12 13 14 15 16 17 18 19

-33686019 0 0 0 97 113 4391232 4391024 4350028 21

类的定义格式:

关键字 **class** **类名** **类名**
私有 **private** :
 成员数据;
 成员函数;
公有 **public** :
 成员数据;
 成员函数;
保护 **protected** :
 成员数据;
 成员函数;
}; **分号不能少**

class Student

```
{ private :  
    char Name[20];  
    float Math;  
    float Chinese;  
  
public :  
    float average;  
    void SetName(char *name);  
    void SetMath(float math);  
    void SetChinese(float ch);  
    float GetAverage(void);  
};
```

用关键字**private**限定的成员称为**私有成员**，对私有成员**限定在该类的内部使用**，即只允许该类中的成员函数使用私有的成员数据，对于私有的成员函数，只能被**该类内的成员函数调用**；类就相当于私有成员的作用域。

用关键字**public**限定的成员称为**公有成员**，
公有成员的数据或函数不受类的限制，**可以在类内或类外自由使用**；对类而言是透明的。

而用关键字**protected**所限定的成员称为**保护成员**，只允许在类内及该类的派生类中使用保护的数据或函数。即保护成员的作用域是**该类及该类的派生类**。

	私有成员	公有成员	保护成员
类内函数	可以调用	可以调用	可以调用
类外函数	不可调用	可以调用	不可调用

	私有函数	公有函数	保护函数
类内函数	可以调用	可以调用	可以调用
类外函数	不可调用	可以调用	不可调用

每一个限制词(**private**等)在类体中可使用多次。一旦使用了限制词，该限制词一直有效，直到下一个限制词开始为止。

如果未加说明，类中成员默认访问权限是**private**，即私有的。

```
class Student
```

```
{
```

均为私有权限

```
    char Name[20];
```

```
    float Math;
```

```
    float Chinese;
```

```
    public :
```

```
        float average;
```

```
        void SetName(char *name);
```

```
        void SetMath(float math);
```

```
        void SetChinese(float ch);
```

```
        float GetAverage(void);
```

均为公有权限

```
};
```


例子：用类实现栈

```
//文件stack.h ， 类Stack的定义
class Stack
{public:
    Stack(int s);
    ~Stack();
    int pop(int &num);           //函数原型
    int push(int num);          //函数原型
private:
    int *data;                  //栈数据存储
    int memNum;                 //栈元素个数
    int size;                   //栈大小
};
```

```
//文件stack.cpp, 类Stack的实现
#include "stack.h"
Stack::Stack(int s)
{data = new int[s];
  size = s;
  memNum = 0;
}
//析构函数
Stack::~~Stack()
{
  delete [] data;
}
```

//弹栈函数，不成功返回0，成功则返回1，
//栈元素由参数返回

```
int Stack::pop(int &num)
{if (memNum==0)
    return 0;
    num = data[--memNum];
    return 1;
}
```

//压栈函数，不成功返回0，成功则返回1

```
int Stack::push(int mem)
{if (memNum == size)
    return 0;
    data[memNum++] = mem;
    return 1;
}
```

```
//文件ex11_2.cpp
#include "stack.h"
#include <iostream.h>
main()
{int i, num;
  Stack myStack(10);
  cout << "Push: ";
  for (i=0; i<11; i++)
    if (!myStack.push(i))
      cout<<endl<<"Push "
        <<i<<" failed."<<endl;
  else
    cout << i << "    ";
```

```
cout << "Pop: ";  
for (i=0; i<11; i++)  
    if (myStack.pop(num) )  
        cout<<num<<"    ";  
    else  
        cout<<endl<<"Pop failed.";  
cout<<endl;  
return 0;  
}
```

程序运行结果:

Push: 0 1 2 3 4 5 6 7 8 9

Push 10 failed.

Pop: 9 8 7 6 5 4 3 2 1 0

Pop failed.

在C++语言中，**结构体类型**只是类的一个特例。结构体类型与类的唯一的区别在于：**在类中，其成员的缺省的存取权限是私有的；而在结构体类型中，其成员的缺省的存取权限是公有的。**

关于类Stack的说明 (1/6)

- 类Stack封装了与栈相关的数据和函数
- 访问说明符public和private定义了类成员（包括数据成员和成员函数）的访问属性
- public: 定义类的公有成员
- private: 定义类的私有成员

关于类Stack的说明 (2/6)

- 定义好的类可以作为新的数据类型来使用

Stack oneStack; //Stack类型的对象

Stack arrayOfStack[10];

 //Stack类型的对象数组

Stack *pStack;

 //Stack类型的指针

Stack &s = oneStack;

 // 引用一个Stack对象

关于类Stack的说明 (3/6)

- 类的定义和实现一般在不同的文件中
 - 类的定义一般放在一个头文件中，供其它需要使用该类的文件包含
 - 类的实现部分一般在一个源文件中，该文件需要包含定义类的头文件
 - 类的定义和实现可以放在一个文件里，但不提倡，因为结构不清晰

关于类Stack的说明 (4/6)

- 类成员函数的实现在类的定义外面时，必须声明其所属，例如：

```
int Stack::pop(int &num)
{ ... }
```

其中Stack为类名，::是作用域运算符

- 类成员函数的实现也可以放在类里，不需要加类名和作用域运算符
- 类的数据成员只能在类里声明，不能在类里初始化

关于类Stack的说明 (5/6)

- 创建类的对象时会自动调用构造函数初始化对象
- 如果构造函数需要参数则声明对象时要提供相应的参数，如
`Stack myStack(10);`
- 系统根据提供的参数列表确定要调用的构造函数

关于类Stack的说明 (6/6)

- 封装和信息隐藏技术使类变得很安全
 - 私有数据成员通过类自己的成员函数访问
 - 只要确保成员函数操作的合理合法性，对象就是安全的
- 接口和实现的分离使得类的实现被独立出来
 - 类的实现部分的修改不会影响到程序的其他部分

例子：用类实现栈

```
//文件stack.h ， 类Stack的定义
class Stack
{public:
    Stack(int s);
    ~Stack();
    int pop(int &num);           //函数原型
    int push(int num);          //函数原型
private:
    int *data;                  //栈数据存储
    int memNum;                 //栈元素个数
    int size;                   //栈大小
};
```

类和对象的定义 (1/12)

■ 数据成员

- 可以定义各种类型的变量、指针、数组等，甚至是其他类的对象
- 在类的定义中数据成员不能赋初值
- 数据成员一般放在`private`部分
- 数据成员也可以放在`public`部分，但不提倡

类和对象的定义 (2/12)

■ 成员函数

- 可以放在public部分（可以被外部函数调用），也可以是私有的（供自己的成员函数使用）
- 成员函数的实现一般放在类定义的外面（也可以在类的里面），并且与类的定义不在一个文件中

类和对象的定义 (3/12)

■ 访问控制

- 三个访问说明符: `public`、`private`和`protected`
- 作用——控制对类的成员的访问
- 在用`class`定义的类中，默认访问方式是`private`
- 在类的定义中，三个访问说明符都可以使用多次（不提倡）

类和对象的定义 (4/12)

■ 访问控制

- 类的私有成员只能被类的成员函数、友元访问
- 类的受保护成员能被类的成员函数、友元和（直接）派生类成员函数、友元访问
- 类的公有成员可以被类的成员函数、友元以及所有能访问到类的对象的外部程序访问
- 类的公有成员是类的对外接口

类和对象的定义 (5/12)

■ 最低访问权原则

- 如果某函数不需要访问某些数据，则不给它访问的权限（如类的私有属性）
- 如果某函数需要读取某数据的值，但不需要修改它，则只给它读的权限（如函数的const参数）
- 只有当函数需要修改某数据时才给它全部的访问权限

类和对象的定义 (6/12)

- 类的对象独立拥有自己的数据成员空间，共享类的成员函数
- 对象之间相互独立，消息是对象之间交互的唯一方式
- 如果一个类的不同对象需要共享数据，怎么办？
- 如果要知道一个类是否有对象存在，怎么办？
- 要知道总共有几个对象是基于某个类的，怎么解决？

类和对象的定义 (7/12)

■ 静态成员

- 类的静态成员包括静态数据成员和静态成员函数
- 类的静态数据成员属于类，被类的所有对象共享
- 类的静态成员可以是私有的、公有的或受保护的

类和对象的定义 (8/12)

■ 静态成员

- 静态数据成员不依赖于对象而存在
- 静态成员函数是操作静态数据成员的函数
- 静态成员函数只能访问类的静态成员，不能调用非静态成员函数、不能访问非静态数据成员
- 非静态成员函数可以访问静态数据成员，也可以调用静态成员函数

类和对象的定义 (9/12)

■ 静态成员

■ 访问静态成员的方式:

对象名 + 点操作符

对象的引用 + 点操作符

对象指针 + 箭头操作符

类名 + 作用域运算符 (::)

例子3: 实现对象计数

```
//文件"stack.h"
class Stack
{public:
    Stack(int s);
    ~Stack();
    int pop(int &num);    //函数原型
    int push(int num);    //函数原型
    static int getObjNum(); //静态成员函数
private:
    int *data;            //栈数据存储
    int memNum;           //栈元素个数
    int size;             //栈大小
    static int objNum;
                        //静态数据成员，记录对象个数
};
```



```
//文件stack.cpp
#include "stack.h"
Stack::Stack(int s)
{data = new int[s];
  size = s;
  memNum = 0;
  objNum++; //一个对象产生时, objNum的值加1
}
Stack::~~Stack()
{delete [] data;
  objNum--;
  //一个对象被撤销时, objNum的值减1
}
```

```
int Stack::pop(int &num)
{if (memNum==0)      return 0;
  num = data[--memNum];
  return 1;
}
int Stack::push(int mem)
{if (memNum == size)  return 0;
  data[memNum++] = mem;
  return 1;
}
//初始化类的静态数据成员
int Stack::objNum = 0;
//实现类的静态成员函数
int Stack::getObjNum()
{ return objNum; }
```

```
//文件ex11_4.cpp
#include "stack.h"
#include <iostream.h>
main()
{cout<<Stack::getObjNum()<<endl;
  {Stack one(10);
    Stack *ptr = &one;
    cout<< ptr-> getObjNum()<<endl;
    {Stack two(10);
      cout<<two.getObjNum() <<endl;
    }
    cout<<one.getObjNum() <<endl;
  }
  cout << Stack::getObjNum() <<endl;
  return 0;
}
```

类和对象的定义 (10/12)

■ 头文件的包含问题

- 头文件会被其他头文件或源文件包含
- 一个源文件可能会多次包含一个头文件

b.h 包含 a.h

c.h 包含 a.h

d.cpp 包含 b.h 和 c.h

- a.h中的内容被重定义

例子4: 圆形类Circle的定义

```
//文件circle.h
#ifndef __CIRCLE__H__
#define __CIRCLE__H__
#include <iostream.h>
class Circle
{public:
    Circle();
    void draw();
private:
    double    x, y;        //x、y坐标
    double    r;           //半径
};
#endif
```

类和对象的定义 (11/12)

- 预处理指令 `#if !defined` 和 `#endif`

`#if !defined __CIRCLE__H__`

判断标识符 `__CIRCLE__H__` 是否被定义，如果没有则编译后续程序，否则跳过后续语句直到指令 `#endif`

- 第一次包含头文件 `circle.h` 时，
`__CIRCLE__H__` 没被定义，类 `Circle` 的定义被编译，`__CIRCLE__H__` 被定义
- 第二次包含头文件 `circle.h` 时，
`__CIRCLE__H__` 已经被定义，类 `Circle` 的定义被跳过，防止重复定义

类和对象的定义 (12/12)

■ 使用类定义对象

- `Circle one; // 声明类Circle的对象one`
- `Circle array[10]; // 声明对象数组array`
- `Circle *ptr = &one; // 声明对象指针`
- `Circle &o = one; // 声明对象引用`
- `Circle *pone, *pten; // 声明两个指针`
- `pone = new Circle; // 动态创建一个对象`
- `pten = new Circle[10]; // 动态创建对象数组`

同类型的对象之间可以整体赋值，这种赋值与对象的成员的访问权限无关。

```
class A {
    float x,y;
public:
    float m,n;
    void Setxy( float a, float b ){ x=a; y=b; }
    void Print(void) { cout<<x<<'\t'<<y<<endl; }
};

void main(void)
{
    A a1,a2;
    a1.m=10; a1.n=20;    //为公有成员数据赋值
    a1.Setxy(2.0,5.0);
    a2=a1;
    a1.Print(); a2.Print();
}
```

同类型的对象之间可以整体赋值

相当于成员数据间相互赋值

对象可以作函数的入口参数（实参、形参），也可以作函数的出口参数。这与一般变量作为函数的参数是完全相同的。

可以定义类类型的指针，类类型的引用，对象数组，指向类类型的指针数组和指向一维或多维数组的指针变量

一个类的对象，可作为另一个类的成员

类作用域、类类型的作用域和对象的作用域

类体的区域称为**类作用域**。类的成员函数与成员数据，其作用域都是属于类的作用域，仅在该类的范围内有效，**故不能在主函数中直接通过函数名和成员名来调用函数。**

```
class A {  
    float x,y;  
public:  
    float m,n;  
    void Setxy( float a, float b ){ x=a; y=b; }  
    void Print(void) { cout<<x<<'\t'<<y<<endl; }  
};
```

```
void main(void)  
{ A a1,a2;  
    a1.m=20; a1.n=10;  
    a1.Setxy(2.0, 5.0);  
    a1.Print();  
}
```

用对象名调用

```
void main(void)  
{ A a1,a2;  
    m=20; n=10;  
    Setxy(2.0, 5.0);  
    Print();  
}
```

不能直接调用

类类型的作用域：在函数定义之外定义的类，其类名的作用域为**文件作用域**；而在函数体内定义的类，其类名的作用域为**块作用域**。

对象的作用域与前面介绍的变量作用域完全相同，**全局对象、局部对象、局部静态对象**等。

```
class A {  
    float x,y;  
public:  
    float m,n;  
    void Setxy( float a, float b ){ x=a; y=b; }  
    void Print(void) { cout<<x<<'\\t'<<y<<endl; }  
}a3,a4;  
void main(void)  
{ A a1,a2;  
    class B{  
        int i,j;  
    public :  
        void Setij(int m, int n){ i=m; j=n; }  
    };  
    B b1,b2;  
    a1.Setxy(2.0, 5.0); b1.Setij(1,2);  
}
```

类A: 文件作用域, 在整个文件中有效

全局对象

局部对象

类B: 块作用域, 在函数内有效。

类的嵌套

在定义一个类时,在其类体中又包含了一个类的完整定义,称为类的嵌套。

类是允许嵌套定义的。

```
class A {
```

```
    class B{
```

```
        int i,j;
```

类B包含在类A中，
为嵌套定义

```
    public :
```

```
        void Setij(int m, int n){ i=m; j=n; }
```

```
};
```

```
    float x,y;
```

```
public:
```

嵌套类的对象

```
    B b1,b2;
```

```
    void Setxy( float a, float b ){ x=a; y=b; }
```

```
    void Print(void) { cout<<x<<'\t'<<y<<endl; }
```

```
};
```

b1 b2 有分配空间吗？

作用域是怎样的？

讲授内容

- 抽象数据类型的概念
- 类的特点和作用
- 类的定义
- 类的访问控制
- 类的数据成员和成员函数
- 类的静态成员
- 类的对象
- 类的构造函数和析构函数
- 类的复合
- **this**指针
- **const**对象/成员函数
- 友元函数
- 友元类

回顾栈的例子..构造函数，析构函数？

```
//文件stack.h ， 类Stack的定义
class Stack
{public:
    Stack(int s);
    ~Stack();
    int pop(int &num);           //函数原型
    int push(int num);           //函数原型
private:
    int *data;                   //栈数据存储
    int memNum;                   //栈元素个数
    int size;                     //栈大小
};
```

```
class Time {  
    int hour = 0;  
    int minute = 0;  
    int sec = 0;  
};
```



这种赋初值的方式正确吗？

```
#include <iostream>

using namespace std;

class Time {
public:
    int hour;
    int minute;
    int sec;
};

Time t1 = {8,20,0};
```

构造函数 (1/8)

- 函数名和类名完全相同
- 没有返回类型
- 在对象建立时被自动调用
- 用于初始化类的对象，保证对象在创建后其数据成员都已经被正确初始化

构造函数 (2/8)

- 构造函数执行的时机
 - **对象**被声明或创建时，构造函数被自动调用
 - 全局对象在所有程序执行之前被创建
 - 局部自动对象在程序执行到声明该对象的语句时被创建
 - 局部静态对象在程序**首次**执行到声明该对象的语句时被创建
 - 动态对象（用new运算符创建）在程序执行到该语句时被创建

构造函数 (3/8)

■ 构造函数重载

- 类的构造函数可以重载
- 重载的构造函数的函数名完全相同，都没有返回类型，但参数列表各不相同
- 类有多个构造函数时，系统根据创建对象时提供的参数来确定调用哪个构造函数

例子5: 重载构造函数

```
//文件circle.h
#ifndef __CIRCLE__H__
#define __CIRCLE__H__
#include <iostream.h>
class Circle
{public:
    Circle( )
    { x = y = r = 5.0; }
    Circle(double a, double b, double c)
    {x = a;
      y = b;
      if (c>0) r = c;
      else r = 5.0;
    }
}
```



```
void draw()  
{cout << "Draw a circle at ("  
    << x << ", "<<y  
    << "), with radius " << r  
    << "." << endl;  
}
```

```
private:
```

```
    double    x, y;    //x、y坐标  
    double    r;       //半径
```

```
};
```

```
#endif
```

构造函数 (4/8)

■ 声明类Circle的对象

```
Circle obj; //不能是Circle obj( );  
//调用第1个构造函数, 对象obj中的x、y、r都  
被赋值为5.0
```

```
Circle obj1(4.0, 5.0, 6.0);  
//调用第2个构造函数, 对象obj中的x、y、r分  
别被赋值为4.0、5.0和6.0
```

构造函数 (5/8)

■ 默认构造函数

- 调用时不需要参数的构造函数是默认构造函数，如
 - 类Circle的第一个构造函数是默认构造函数
 - 所有参数都是默认参数的构造函数是默认构造函数
- 当一个类没有定义任何构造函数时，系统为该类自动生成默认构造函数 `类名() {}`
- 一个类至多拥有一个默认构造函数

例子6: 用默认参数修改例子5

```
//文件circle.h
#ifndef __CIRCLE__H__
#define __CIRCLE__H__
#include <iostream>
class Circle
{public:
    //带默认参数的构造函数
    Circle(double a = 5.0, double b = 5.0, double c = 5.0)
    { x = a;
      y = b;
      if (c>0) r = c;
      else r = 5.0;
    }
}
```

```
void draw()  
    {cout << "Draw a circle at ("  
        << x << ", "<<y  
        << "), with radius " << r  
        << "." << endl;  
    }  
private:  
    double    x, y;    //x、y坐标  
    double    r;       //半径  
};  
#endif
```

构造函数 (6/8)

- 使用例子6中的类Circle声明对象

```
Circle a;
```

```
//相当于      Circle a(5.0, 5.0, 5.0);
```

```
Circle b(1.0);
```

```
//相当于      Circle b(1.0, 5.0, 5.0);
```

```
Circle c(3.0, 4.0);
```

```
//相当于      Circle c(3.0, 4.0, 5.0);
```

构造函数 (7/8)

■ 复制构造函数

- 根据类的某个对象复制出一个内容完全相同的新对象的构造函数叫复制构造函数
- 复制构造函数的参数是该类的对象（传引用的方式）
- 如果类定义时没有定义复制构造函数，编译时会自动加入一个完成数据成员复制的复制构造函数
- 当参数传对象值或函数返回对象值时自动调用复制构造函数

例子7: 为例子6增加一个复制构造函数

```
//文件circle.h
#ifndef __CIRCLE__H__
#define __CIRCLE__H__
#include <iostream.h>
class Circle
{public:
    //带默认参数的构造函数
    Circle(double a = 5.0, double b = 5.0, double c = 5.0)
    { x = a;
      y = b;
      if (c>0) r = c;
      else r = 5.0;
    }
}
```


//复制构造函数

```
Circle(Circle &c)
```

```
{    x = c.x;
```

```
    y = c.y;
```

```
    r = c.r;
```

```
}
```

```
void draw()
```

```
{cout << "Draw a circle at ("
```

```
    << x << ", " << y
```

```
    << "), with radius " << r
```

```
    << "." << endl;
```

```
}
```

```
private:
```

```
    double    x, y;    //x、y坐标
```

```
    double    r;      //半径
```

```
};
```

```
#endif
```

构造函数 (8/8)

- 使用复制构造函数

```
Circle a (3.0, 4.0, 5.0);
```

```
Circle b(a);
```

- 当对象中包含指向动态空间的指针数据成员时，复制构造函数的实现不能只是简单的赋值

- 应该先为当前对象分配新的空间
- 再将参数对象中相应的数据复制过来

深拷贝，浅拷贝

复制构造函数：总结1

■ 定义

- 复制构造函数是用于创建一个新对象，该对象与现有对象具有相同值的构造函数。
- 它采用一个与自身类型相同的对象作为参数。
- 创建新对象的副本，而不共享原始对象中的数据。
- 执行逐位复制，这意味着它复制原始对象的所有成员变量。

■ 默认复制构造函数

- 如果未定义显式的复制构造函数，编译器将生成一个默认复制构造函数。
- 它执行逐位复制，但对于包含指针或引用成员类，可能导致浅拷贝。

复制构造函数：总结2

■ 赋值操作符重载

- 在执行赋值操作时，编译器默认调用复制构造函数，除非重载了赋值操作符 (=)。
- 重载赋值操作符可确保对象的内容正确复制，并可处理浅拷贝问题。

■ 最佳实践

- 定义显式的复制构造函数以控制对象复制的行为。
- 在复制构造函数中注意指针和引用成员的处理。对于包含指针或引用成员类，实现深拷贝。
- 重载赋值操作符以确保对象内容正确复制。

回顾栈的例子..构造函数，析构函数？

```
//文件stack.h ， 类Stack的定义
class Stack
{public:
    Stack(int s);
    ~Stack();
    int pop(int &num);           //函数原型
    int push(int num);           //函数原型
private:
    int *data;                   //栈数据存储
    int memNum;                  //栈元素个数
    int size;                    //栈大小
};
```

析构函数 (1/3)

- 析构函数名是字符 ‘~’ 加上类名
- 析构函数不能指定返回类型
- 没有参数
- 不能被重载
- 当一个类没有定义任何析构函数时，系统为该类自动生成析构函数 `~类名() {}`
- 其作用是在撤销对象前为对象做最后的清理工作
 - 一般为释放对象在生存期内动态分配的空间

11.5 析构函数 (2/3)

- 析构函数执行的时机
 - 对象被撤销时，类的析构函数被自动调用
 - 全局对象在所有程序执行完毕之后被撤销（执行完main函数最后一条语句之后）
 - 局部自动对象在退出该对象的作用域时被撤销

11.5 析构函数(3/3)

- 析构函数执行的时机
 - 对象被撤销时，类的析构函数被自动调用（续）
 - 局部静态对象在程序执行完毕后或程序执行 `exit` 语句之后，其析构函数被执行
 - 局部静态对象的析构函数在全局对象的析构函数执行之前执行
 - 动态创建对象的析构函数在执行相应的 `delete` 操作时执行

例子8：构造函数和析构函数的执行时机

```
//文件createanddestroy.h
#ifndef __CREATEANDDESTROY__H__
#define __CREATEANDDESTROY__H__
#include <iostream.h>
class CreateAndDestroy
{public:
    CreateAndDestroy(int n)
    {no = n;
        cout<<"Object "
            <<no<<" created!"<<endl;}
    ~CreateAndDestroy()
    {cout<<"Object "
        <<no<<" destructed!"<<endl;}
private:
    int no; }; #endif
```

```
//文件ex11_6.cpp
#include "createanddestroy.h"
CreateAndDestroy one(1); //定义全局对象one
main()
{
    CreateAndDestroy *ptr;
    CreateAndDestroy two(2);
        //定义局部自动对象two
    {
        CreateAndDestroy three(3);
            //定义局部自动对象three
        ptr = new CreateAndDestroy(4);
            //动态创建对象，由ptr指向
        static CreateAndDestroy five(5);
            //定义局部静态对象five
    }
    delete ptr; //释放动态创建的对象
    return 0;
}
```

程序运行结果:

```
Object 1 created!  
Object 2 created!  
Object 3 created!  
Object 4 created!  
Object 5 created!  
Object 3 destructed!  
Object 4 destructed!  
Object 2 destructed!  
Object 5 destructed!  
Object 1 destructed!
```

类的复合

- 软件重用的一种形式
- 一个类可以把其它类的对象作为自己的数据成员
- 大对象包含一个或多个小对象

例子1: 利用类的复合定义圆柱类Column

```
//文件circle.h
#ifndef __CIRCLE__
#define __CIRCLE__
#include <iostream>
class Circle
{public:
    Circle(double a = 5.0,
           double b = 5.0, double c = 5.0);
    ~Circle();
    double area();
    void print();
private:
    double x, y; //x、y坐标
    double r;     //半径
}; #endif
```

```
//文件circle.cpp
#include "circle.h"
#define PI 3.14159
Circle::Circle(double a,
                double b, double c)
{
    x = a;
    y = b;
    if (c>0) r = c;
    else     r = 5.0;
    cout<<"Circle object start: x = "<<x
         <<" , y = "<<y<<" , r = "<<r<<endl;
}
Circle::~~Circle()
{
    cout<<"Circle object end : x = "<<x
         <<" , y = "<<y<<" , r = "<<r<<endl;
}
```

```
double Circle::area()  
{  
    return  PI*r*r;  
}
```

```
void Circle::print()  
{  
    cout<<"x = "<<x<<" , y = "<<y  
        <<" , r = "<<r;  
}
```

```
//文件column.h
#ifndef __COLUMN__H__
#define __COLUMN__H__
#include "circle.h"
class Column
{public:
    Column(double h=5.0, double a=5.0,
           double b=5.0, double c=5.0);
    ~Column();
    double volume();
private:
    Circle circle;    // 数据成员circle
    double height;    // 数据成员height
};
#endif
```



```
//文件column.cpp
#include "column.h"
Column::Column(double h, double a,
               double b, double c )
:circle(a,b,c) //为数据成员circle提供初值
{if (h>0)      height = h;
else          height = 5.0;
cout<<"Column object start: height = "
    <<height<<" , ";
circle.print();
cout<<endl;
}
```

//多个数据成员为对象?

```
Column::~~Column()  
{cout<<"Column object end: height = "  
    <<height<<" , ";  
    circle.print(); cout<<endl;  
}
```

```
double Column::volume()  
{  
    return height*circle.area();  
}
```

//数据成员对象中数据的访问?

```
//文件ex12_1.cpp
//使用类Column
#include "column.h"
#include <iostream.h>
main()
{Column obj(2.3, 3.4, 4.5, 5.6);
  cout<<"The volume of obj is "
        <<obj.volume()<<endl;
  return 0;
}
```

程序执行结果:

Circle object start: $x=3.4, y=4.5, r=5.6$

Column object start:

$height=2.3, x=3.4, y=4.5, r=5.6$

The volume of obj is 226.597

Column object end:

$height=2.3, x=3.4, y=4.5, r=5.6$

Circle object end: $x=3.4, y=4.5, r=5.6$

- 多个数据成员是其他类的对象，如何初始化？
- 数据成员对象中数据的访问？

```
#include <iostream>
using namespace std;

// 假设有一个名为InnerClass的类
class InnerClass {
public:
    InnerClass(int value) : data(value) {}
    // 假设InnerClass有一个带有参数的构造函数
    int getData() { return data; }
private:
    int data;
};
```

// 假设有一个名为OuterClass的类，其中有
InnerClass对象作为数据成员

```
class OuterClass {
public:
    OuterClass(int value1, int value2) :
        innerObj1(value1), innerObj2(value2) {
        // 其他初始化操作
    }

    void displayData() {
        cout << "Inner object 1 data: " <<
            innerObj1.getData() << endl;
        cout << "Inner object 2 data: " <<
            innerObj2.getData() << endl;
    }

private:
    InnerClass innerObj1;
    InnerClass innerObj2;
};
```

```
int main() {  
    OuterClass obj(10, 20);  
    obj.displayData();  
    return 0;  
}
```

```
OuterClass(int value1, int value2) :  
    innerObj1(value1), innerObj2(value2) {  
    // 其他初始化操作  
}
```




福州大学
FUZHOU UNIVERSITY

数据成员对象中数据的访问？

```
class InnerClass {  
public:  
    int data; // 内部数据成员  
};
```

```
class OuterClass {  
public:  
    InnerClass innerObj; // 内部对象  
  
    void setData(int value) {  
        innerObj.data = value; // 访问内部对象数据成员  
  
    int getData() {  
        return innerObj.data; // 返回内部对象的数据成员  
    };  
};
```

不同对象占据内存中的不同区域，它们所保存的数据各不相同，但对成员数据进行操作的成员函数的程序代码均是一样的。

对象的定义格式：

《存储类型》类名 对象1，对象2 《,.....》；

Student st1, st2;



类名



对象名

在建立对象时，只为对象分配用于保存数据成员的内存空间，而成员函数的代码为该类的每一个对象所共享。

this指针 (1/5)

- 一个类的多个对象各自拥有类的数据成员的一个副本
- 类的成员函数则被其所有对象共享
- 一个类的不同对象响应相同的消息时，调用的是同一个函数
- 问题：成员函数如何知道它要操作哪个对象的数据？

this指针 (2/5)

- 每个对象都有一个指向自身的this指针
- 对象调用成员函数时会将自己的this指针传递给成员函数（隐含参数）
- 成员函数访问对象的所有数据成员都是通过this指针访问，如

```
double Column::volume (Column *this)
{
    return this-> height
           * this-> circle.area();
}
// this指针可以在成员函数中显式使用
```

例子2: 用this指针访问对象的数据成员 (1/2)

```
//文件ex12_2.cpp
#include <iostream>
class Test
{public:
    Test(int n = 0);
    void print();
private:
    int data;
};
Test::Test(int n)
{data = n;}
void Test::print()
{cout<<"data = "<<this->data<<"!"<<endl;
}
```



例子2: 用this指针访问对象的数据成员 (2/2)

```
main()  
{Test obj1(10), obj2(20);  
  cout<<"obj1: ";  
  obj1.print();  
  cout<<"obj2: ";  
  obj2.print();  
  return 0;  
}
```

程序执行结果:

obj1: data = 10!

obj2: data = 20!

this指针 (3/5)

- 成员函数访问数据成员可以直接访问（构造函数Test），也可以通过this指针访问（函数print）
- 实际上都是通过this指针
- this指针的另外一个作用——访问对象自身（*this）

例子3: 实现成员函数的连续调用 (1/2)

```
#include <iostream>
class Test
{public:
    Test(int n = 0);
    Test & setData(int n);
    void print();
private:
    int data;
};
Test::Test(int n)
{data = n;
}
```

例子3: 实现成员函数的连续调用 (2/2)

```
Test & Test::setData(int n)
{data = n;
  return *this;
}
void Test::print()
{cout<<"data = "<<data<<"!"<<endl;
}
main()
{Test obj1;
  cout<<"obj1: ";
  obj1.setData(100).print();
  return 0;
}
```

this指针 (4/5)

- 成员函数setData返回类型是Test &, 返回值是*this
- 一个对象调用成员函数setData将获得该对象的引用——连续调用的关键

`obj1.setData(100).print();` 正确

`obj1.print().setData(100);` 错误, 为什么?

this指针 (5/5)

- 类的静态成员函数为什么不能访问非静态数据成员？
 - 静态成员函数（属于类）没有维护this指针
 - 访问非静态数据成员（属于对象）需要this指针
- 静态成员函数为什么不能调用非静态成员函数
 - 调用非静态成员函数需要一个隐含参数(this指针)
 - 静态成员函数没有
- 非静态成员函数可以调用静态成员函数吗？为什么？

非静态成员函数可以调用静态成员函数

```
class MyClass {
public:
    static void staticFunction() {
        std::cout << "static function." <<
std::endl;
    }
    void nonStaticFunction() {
        std::cout << "non-static function." <<
std::endl;
        staticFunction(); //调用静态成员函数
    };
};

int main() {
    MyClass obj;
    obj.nonStaticFunction();
    return 0; }
```



福州大学
FUZHOU UNIVERSITY

this 指针

不同对象占据内存中的不同区域，它们所保存的数据各不相同，但对成员数据进行操作的成员函数的程序代码均是一样的。

a1.Setxy(1,2);

a2.Setxy(3,4);

```
class A{
```

```
    int x,y;
```

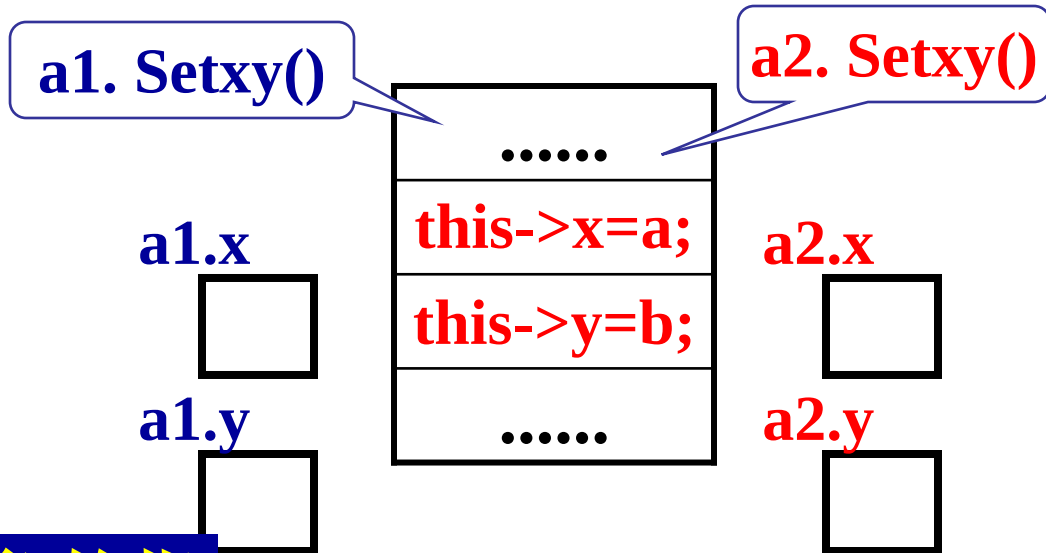
```
    public:
```

```
    void Setxy(int a, int b)
```

```
    { x=a; y=b;}
```

```
};
```

```
A  a1, a2;
```



系统自动将对象的指针带到成员函数中

当对一个对象调用成员函数时，编译程序先将对象的地址赋给**this**指针，然后调用成员函数，每次成员函数存取数据成员时，也隐含使用**this**指针。

this指针具有如下形式的缺省说明：

类名

Stu *const this;

即 **this** 指针里的地址是一个常量

this指针显式使用



```
class Point {  
private:  
    int x, y;  
public:  
    Point(int x = 0, int y = 0) {  
        this->x = x;  
        this->y = y;  
    }  
    void setX(int x) {  
        this->x = x;  
    }  
    void setY(int y) {  
        this->y = y;  
    }  
    void print() {  
        cout << "Point(" << this->x << ", " << this->y << ")" << endl;  
    }  
};
```

友元函数和友元类 (1/3)

- 类的数据成员都设为私有的——良好的程序设计风格
- 如果某个（些）外部函数需要直接访问类的数据成员，怎么办？
 - 方法一：数据公有
 - 方法二：友元

友元函数和友元类 (2/3)

■ 友元函数

- 友元函数是一般函数或其它类的成员函数
- 友元函数具有访问类的所有成员的权限
- 声明友元函数的方法: friend + 函数原型
- 除非能带来极大的便利, 否则不要使用友元

例子5: 使用友元函数修改类的私有数据成员

```
//文件triangle.h
#ifndef __TRIANGLE__H__
#define __TRIANGLE__H__
class Triangle
{friend void setA(Triangle &t, int n);
public:
    Triangle(int x = 5,
              int y = 5, int z = 5);
    void print();
private:
    int a, b, c;
};
#endif
```

```
//文件triangle.cpp
#include "triangle.h"
#include <iostream.h>
Triangle::Triangle(int x, int y, int z)
{if (x+y>z && x+z>y && y+z>x)
    { a = x;  b = y;  c = z;}
  else
    { a = b = c = 5; }
}
void Triangle::print( )
{cout<<"Triangle: "<<a<<" ,  "
    << b <<" ,  "<<c<<endl;
}
```

```
//文件ex12_5.cpp
#include "triangle.h"
void setA(Triangle &t, int n)
{t.a = n;          //访问对象t的私有成员a
}
```

```
main()
{Triangle t;
  t.print();
  setA(t, 10);
  t.print();
  return 0;
}
```



友元函数和友元类 (3/3)

■ 友元类

- 一个类的友元类的所有成员函数都有访问类的所有成员的权限
- 友元类的声明方法: `friend + class + 类名`
- 友元类的声明是单方面的
- 友元类的声明不是传递的

例子6: 友元类 (1/3, 方案1)

```
//文件class.h
#ifndef __CLASS_H__
#define __CLASS_H__
class B;
class A
{public:
    void setB(B &b, int m);
    void print(B &b);
};
class B
{friend class A;           //声明友元类
private:
    int data;
};
#endif
```

例子6: 友元类 (1/3, 方案2)

```
//文件class.h
#ifndef __CLASS_H__
#define __CLASS_H__
class B
{friend class A;           //声明友元类
 private:
    int data;
};
class A
{public:
    void setB(B &b, int m);
    void print(B &b);
};
#endif
```

问题: 如何指定A类的成员函数setB为B类的友元函数?

例子6: 友元类 (2/3)

```
//文件class.cpp
//实现类A和B
#include "class.h"
#include <iostream>
void A::setB(B &b, int m)
{b.data = m;           //访问B的私有成员
}
void A::print(B &b)
{
//访问B的私有成员
cout<<"The private data of class B: "
    <<b.data<<endl;
}
```

例子6: 友元类 (3/3)

```
//文件ex12_6.cpp
#include "class.h"
main()
{A a;
  B b;
//调用类A的成员函数修改类B的对象b的私有数据
  a.setB(b, 10);
//调用类A的成员函数访问类B的对象b的私有数据
  a.print(b);
  return 0;
}
```

共用数据保护

■ 常对象

- 必须要有初值，且该对象生命周期中所有数据成员的值都不能修改。
- 定义常对象
 - `Stack const myStack(10);` 或
 - `const Stack myStack(10);`
- 只能调用其**常成员函数**（防止普通成员函数会修改常对象中数据成员的值）

共用数据保护

■ 常成员函数

- 可以访问常对象的数据成员，但不允许修改。
- 定义常成员函数
 - `int getNum() const;`
- 如果一个对象被声明为常对象，只能调用其常成员函数（防止普通成员函数会修改常对象中数据成员的值）

共用数据保护

- 常对象只能调用常成员函数，不能调用普通成员函数
- 常成员函数不能修改对象的数据成员
- 以上两点保证了常对象的数据成员不会被改变。

共用数据保护

■ 常数据成员

- 只能通过构造函数初始化，之后其值不能改变。
- 定义常数据成员
 - `const int size;`
- 如果一个对象被声明为常对象，只能调用其常成员函数（防止普通成员函数会修改常对象中数据成员的值）

共用数据保护

- `Time * const ptr1;`
 - 常指针变量，只能指向某个对象，不能改变指向
- `const Time *ptr2;`
 - 指向常变量的指针，一般用于指向常变量（可以指向非常变量，此时不能通过指针改变该变量的值）

const 特性 (1/3)

- 对象应用的最低访问权原则
 - 有些对象不能被修改——让它不能被修改
 - 有些成员函数不需要修改对象的数据成员——不让它修改数据成员
 - const 对象——不能被修改的对象
 - const 成员函数——不能修改数据成员的成员函数

const 特性 (2/3)

- 声明const对象

```
const Circle baseCircle(0, 0, 1);
```

- 声明const成员函数

```
void Test::print( ) const  
{    cout<<"data = "<<data<<"!\n"; }
```

- 声明const数据成员

- 和声明普通const变量的方法相同（初始化方式不同）

const特性 (3/3)

■ const约束

- const对象不能调用非const成员函数
- const成员函数不能调用非const成员函数
- 非构造函数、析构函数不能修改const对象

例子4: 初始化const数据成员的方法

```
class Circle
{public:
    Circle(double a = 5.0,
           double b = 5.0, double c = 5.0);
private:
    double x, y; //x、y坐标
    const double r; //半径
};
Circle::Circle(double a,
               double b, double c)
:r(c) //const数据成员的初始化
{
    x = a;
    y = b;
}
```

const 数据成员？ 静态数据成员？

■ Const 数据成员

- const 数据成员是指在类中通过const关键字声明的成员变量，一旦其值被初始化之后就不能再被修改。
- 每个对象都有自己的const数据成员副本，因为const数据成员的值是与对象绑定的，它们在每个对象中都有自己的一份拷贝。

■ 静态数据成员

- 静态数据成员是类的所有对象共享的成员变量，它的值对所有类的对象来说是相同的。
- 静态数据成员只有一份拷贝，它属于类而不属于任何特定的对象实例。

学习目的检测

- 了解抽象数据类型的概念
- 掌握类的定义方法
- 掌握类的成员函数的定义方法
- 掌握类的静态成员的作用及使用方法
- 掌握如何在类的定义中使用访问控制策略
- 掌握构造函数、析构函数的定义方法、作用及执行时机

学习目的检测

- 掌握利用复合机制编写较复杂类的方法
- 理解this指针的作用、原理，了解类和对象的实现机制
- 掌握使用this指针编写能连续调用的成员函数的方法
- 掌握const对象和const成员函数的定义和使用方法
- 掌握编写友元函数的方法
- 掌握友元类的用法

构造函数、析构函数： 补充

构造函数和析构函数是在类体中说明的两种特殊的成员函数。

构造函数是在创建对象时，使用给定的值来将对象初始化。

析构函数的功能正好相反，是在系统释放对象前，对对象做一些善后工作。

构造函数可以带参数、可以重载，同时没有返回值。

构造函数是类的成员函数，系统约定构造函数名必须与类名相同。构造函数提供了初始化对象的一种简单的方法。

对构造函数，说明以下几点：

1. 构造函数的函数名必须与类名相同。构造函数的主要作用是完成初始化对象的数据成员以及其它的初始化工作。
2. 在定义构造函数时，不能指定函数返回值的类型，也不能指定为void类型。
3. 一个类可以定义若干个构造函数。当定义多个构造函数时，必须满足函数重载的原则。

4.构造函数可以指定参数的缺省值。

5.若定义的要声明该类的对象时，构造函数必须是**公有的成员函数**。如果定义的类型仅用于派生其它类时，则可将构造函数定义为**保护的成员函数**。

由于构造函数属于类的成员函数，它对私有数据成员、保护的数据成员和公有的数据成员**均能进行初始化**。

对局部对象，静态对象，全局对象的初始化

对于局部对象，每次定义对象时，都要调用构造函数。

对于静态对象，是在首次定义对象时，调用构造函数的，且由于对象一直存在，**只调用一次构造函数**。

对于全局对象，是**在main函数执行之前调用构造函数的**。

```
class A
```

```
{  int x,y;
```

```
    public:
```

```
        A(int a){  x=a; cout<<"1\n";}
```

```
        A(int a, int b) {  x=a,y=b;cout<<"2\n";}
```

```
};
```

```
A  a1(3);
```

1

```
void f(void) {  A  b(2,3);}
```

2

```
void main(void)
```

2

```
{  A  a2(4,5);
```

2

```
    f(); f();
```

```
}
```

```
class A{
    float x,y;
public:
    A(float a, float b){x=a;y=b;cout<<"初始化自动局部对象\n";}
    A(){ x=0; y=0; cout<<"初始化静态局部对象\n";}
    A(float a){ x=a; y=0; cout<<"初始化全局对象\n"; }
    void Print(void){ cout<<x<<"\t"<<y<<endl; }
};
```

A a0(100.0);//定义全局对象

void f(void)

{ cout<<" 进入f()函数\n";

A a2(1,2);

static A a3; //初始化局部静态对象 }

void main(void)

{ cout<<"进入main函数\n";

A a1(3.0, 7.0);//定义局部自动对象

f(); f(); }

初始化全局对象

进入main函数

初始化自动局部对象

进入f()函数

初始化自动局部对象

初始化局部静态变量

进入f()函数

初始化自动局部对象

缺省的构造函数

在定义类时，若没有定义类的构造函数，则编译器**自动**产生一个缺省的构造函数，其格式为：

```
className::className() { }
```

缺省的构造函数并不对所产生对象的数据成员赋初值；即**新产生对象的数据成员的值是不确定的**。

```
class A{
    float x,y;
public:
    A(){ }//缺省的构造函数，编译器自动产生,可以不写
    float Sum(void) { return x+y; }
    void Set(float a,float b) { x=a; y=b;}
    void Print(void) {cout<<"x="<<x<<"\t"<<"y="<<y<<endl; }
};

void main(void)
{
    A a1,a2;//产生对象时，自动调用缺省的构造函数，不赋值
    a1.Set (2.0,4.0);
    cout<<"a1: ";
    a1.Print ();
    cout<<"a1.sum="<<a1.Sum ()<<endl;
    a2.Print();//打印随机值
}
```

关于缺省的构造函数，说明以下几点：

- 1、在定义类时，只要显式定义了一个类的构造函数，则编译器就不产生缺省的构造函数
 - 2、所有的对象在定义时，必须调用构造函数
- 不存在没有构造函数的对象！

```
class A{
```

```
    float x,y;
```

显式定义了构造函数，不
产生缺省的构造函数

```
public:
```

```
    A(float a,float b)    {    x=a; y=b; }
```

```
    void Print(void){    cout<<x<<'\\t'<<y<<endl; }
```

```
};
```

```
void main(void)
```

```
{    A a1;
```

error,定义时，没有
构造函数可供调用

```
    A a2(3.0,30.0);
```

```
}
```

3、在类中，若定义了没有参数的构造函数，或各参数均有缺省值的构造函数也称为缺省的构造函数，缺省的构造函数只能有一个。

4、产生对象时，系统必定要调用构造函数。所以任一对象的构造函数必须唯一。

```
class A{  
    float x,y;  
public:  
    A(float a=10,float b=20){    x=a; y=b; }  
    A(){ }  
    void Print(void){    cout<<x<<'\t'<<y<<endl; }  
};  
void main(void)  
{  
    A a1;  
    A a2(3.0,30.0);  
}
```

两个函数均为缺省的构造函数

错误：两个构造函数均可供调用，构造函数不唯一

构造函数与new运算符

可以使用new运算符来动态地建立对象。建立时要自动调用构造函数，以便完成初始化对象的数据成员。最后返回这个动态对象的起始地址。

用new运算符产生的动态对象，在不再使用这种对象时，必须用delete运算符来释放对象所占用的存储空间。

用new建立类的对象时，可以使用参数初始化动态空间。

```
class A{  
    float x,y;  
public:  
    A(float a, float b)    {    x=a;y=b;    }  
    A()    {    x=0; y=0;    }  
    void Print(void)    { cout<<x<<'\\t'<<y<<endl; }  
};  
void main(void)  
{ A *pa1,*pa2;  
    pa1=new A(3.0, 5.0);//用new动态开辟对象空间，初始化  
    pa2=new A;//用new动态开辟空间，调用构造函数初始化  
    pa1->Print();  
    pa2->Print();  
    delete pa1; //用delete释放空间  
    delete pa2; //用delete释放空间  
}
```

3 5

0 0


```
class A{
    float x,y;
public:
    A(float a,float b)
{
    x=a;y=b;cout<<"调用非缺省的构造函数\n";}
    A() { x=0; y=0; cout<<"调用缺省的构造函数\n" ;}
    ~A() { cout<<"调用析构函数\n";}
    void Print(void) { cout<<x<<"\t"<<y<<endl; }
};

void main(void)
{
    A a1;
    A a2(3.0,30.0);
    cout<<"退出主函数\n";
}
```

调用缺省的构造函数

调用非缺省的构造函数

退出主函数

调用析构函数

调用析构函数

在程序的执行过程中，对象如果用**new**运算符开辟了空间，则在类中**应该**定义一个析构函数，并在析构函数中使用**delete**删除由**new**分配的内存空间。因为在撤消对象时，系统自动收回为对象所分配的存储空间，而不能自动收回由**new**分配的动态存储空间。

```

class Str{
    char *Sp;   int Length;
public:
    Str(char *string)
    {   if(string){           Length=strlen(string);
                                Sp=new char[Length+1];
                                strcpy(Sp,string);
                            }
        else   Sp=0;
    }
    void Show(void){ cout<<Sp<<endl; }
    ~Str(){ if(Sp)   delete []Sp; }
};

void main(void)
{   Str s1("Study C++");
    s1.Show();
}
    
```

在构造函数中将成员数据指针指向动态开辟的内存

用初值为开辟的内存赋值

析构函数，当释放对象时收回用new开辟的空间

'S'	't'	'u'	'd'	'y'	' '	'C'	'+'	'+'	'\0'
-----	-----	-----	-----	-----	-----	-----	-----	-----	------



string

'S'	't'	'u'	'd'	'y'	' '	'C'	'+'	'+'	'\0'
-----	-----	-----	-----	-----	-----	-----	-----	-----	------



Sp

new开辟的空间

Length=strlen(string);

Sp=new char[Length+1];



strcpy(Sp,string);



可以用**new**运算符为对象分配存储空间，如：

A *p;

p=new A;

这时必须用**delete**才能释放这一空间。

delete p;

用**new**运算符为对象分配动态存储空间时，调用了构造函数，用**delete**删除这个空间时，调用了析构函数。当使用运算符**delete**删除一个由**new**动态产生的对象时，它首先调用该对象的析构函数，然后再释放这个对象占用的内存空间。

```
class A{
    float x,y;
public:
    A(float a, float b){ x=a; y=b; cout<<"调用了构造函数\n";}
    void Print(void){ cout<<x<<"\t"<<y<<endl; }
    ~A() { cout<<"调用了析构函数\n"; }
};

void main(void)
{ cout<<"进入main()函数\n";
    A *pa1;
    pa1=new A(3.0, 5.0);//调用构造函数
    pa1->Print();
    delete pa1; //调用析构函数
    cout<<"退出main()函数\n";
}
```

进入main()函数

调用了构造函数

3 5

调用了析构函数

退出main()函数

不同存储类型的对象调用构造函数及析构函数

- 1、**对于全局定义的对象**（在函数外定义的对象），在程序开始执行时，调用构造函数；到程序结束时，调用析构函数。
- 2、**对于局部定义的对象**（在函数内定义的对象）当程序执行到定义对象的地方时，调用构造函数；在退出对象的作用域时，调用析构函数。
- 3、**用static定义的局部对象**，在首次到达对象的定义时调用构造函数；到程序结束时，调用析构函数

4、对于用**new**运算符动态生成的对象，在产生对象时调用构造函数，只有使用**delete**运算符来释放对象时，才调用析构函数。若不使用**delete**来撤消动态生成的对象，程序结束时，对象仍存在，并占用相应的存储空间，即系统不能自动地调用析构函数来撤消动态生成的对象。


```

class A{
    float x,y;
public:
    A(float a, float b){x=a;y=b;cout<<"初始化自动局部对象\n";}
    A(){ x=0; y=0; cout<<"初始化静态局部对象\n";}
    A(float a){ x=a; y=0; cout<<"初始化全局对象\n"; }
    ~A(){ cout<<"调用析构函数" <<endl; }
};

A a0(100.0);//定义全局对象
void f(void)
{ cout<<" 进入f()函数\n";
    A ab(10.0, 20.0);//定义局部自动对象
    static A a3; //初始化局部静态对象
}

void main(void)
{ cout<<"进入main函数\n";
  f();  f(); }
    
```

初始化全局对象
 进入main函数
 进入f()函数
 初始化自动局部对象
 初始化静态局部对象
 调用析构函数
 进入f()函数
 初始化自动局部对象
 调用析构函数
 调用析构函数
 调用析构函数

动态构造及析构对象数组

用**new**运算符来动态生成对象数组时，自动调用构造函数，而用**delete**运算符来**释放p1所指向的对象数组**占用的存储空间时，在指针变量的前面必须加上**[]**，才能将数组元素所占用的空间全部释放。否则，只释放第**0**个元素所占用的空间。

```
pa1=new A[3];
```

```
....
```

```
delete [ ]pa1;
```

```
class A{
    float x,y;
public:
    A(float a=0, float b=0){x=a; y=b;cout<<"调用了构造函数\n";}
    void Print(void){ cout<<x<<'\\t'<<y<<endl; }
    ~A() { cout<<"调用了析构函数\n"; }
};

void main(void)
{ cout<<"进入main()函数\n";
  A *pa1;
  pa1=new A[3];//开辟数组空间
      cout<<"\\n完成开辟数组空间\\n\\n";
  delete [] pa1; //必须用[]删除开辟的空间
  cout<<"退出main()函数\n";
}
```

进入main()函数

调用了构造函数

调用了构造函数

调用了构造函数

完成开辟数组空间

调用了析构函数

调用了析构函数

调用了析构函数

退出main()函数⁶⁸

缺省的析构函数

若在类的定义中没有显式地定义析构函数时，则编译器自动地产生一个缺省的析构函数，其格式为：

```
ClassName::~~ClassName() { };
```

任何对象都必须有构造函数和析构函数，但在撤消对象时，要释放对象的数据成员用**new**运算符分配的动态空间时，必须显式地定义析构函数。

实现类型转换的构造函数

同类型的对象可以相互赋值，相当于类中的数据成员相互赋值；

如果**直接将数据赋给对象**，所赋入的数据需要强制类型转换，**这种转换需要调用构造函数**。

```

class A{
    float x,y;
public:
    A(float a,float b) {x=a;y=b;cout<<"调用构造函数\n";}
    ~A()      {      cout<<"调用析构函数\n";}
    void Print(void) {  cout<<x<<"\t"<<y<<endl; }
};

void main(void)
{ A  a1(1.0, 10.0);
  a1.Print();
  a1=A(3.0 , 30.0);
  a1.Print();
  cout<<"退出主函数\n";
}

```

调用构造函数
1 10
调用构造函数
调用析构函数
3 30
退出主函数
调用析构函数

产生临时对象，
初始化并赋值后
立即释放

注意：当构造函数只有一个参数时，可以用=强制赋值

```
class B{
    float x;
public:  B(float a) {x=a; cout<<"调用构造函数\n";}
        ~B() { cout<<"调用析构函数\n";}
        void Print(void) { cout<<x<<endl; }
};

void main(void)
{B b1(1.0);b1.Print();
  B b2=100;
  b2.Print();
  b1=10;      b1.Print();
  cout<<"退出主函数\n";
}
```

单参数可以这样赋值

b1=B(10)

产生一个
临时对象

调用构造函数

1

调用构造函数

100

调用构造函数

调用析构函数

10

退出主函数

调用析构函数

调用析构函数

Thank you !